
NEUROSymbolic EMBODIED AGENTS

A PREPRINT

Mohammad Albinhassan*

Department of Computing
Imperial College London
m.albinhassan23@imperial.ac.uk

Yuming Feng*

Department of Computer Science
Johns Hopkins University, Whiting School of Engineering
yfeng97@jh.edu

Alessandra Russo

Department of Computing
Imperial College London
a.russo@imperial.ac.uk

Pranava Madhyastha

Department of Computer Science
City, University of London
pranava.madhyastha@city.ac.uk

ABSTRACT

Language and vision-language models generate plausible embodied plans but do not guarantee executability, as their outputs can violate environment dynamics or act on incorrectly grounded entities. We present a neurosymbolic agent that factors long-horizon household tasks into task-directed visual exploration and constrained symbolic planning. In the first phase, a vision-language model and exploration harness acquire goal-relevant predicates and instance bindings from egocentric observations and grounded interactions, producing a symbolic initial state. In the second, a PDDL transition model restricts decoding to tokens that extend applicable actions. Monte Carlo tree search then evaluates executable continuations using a domain-independent planning heuristic. The resulting plans are executable by construction under the transition model, with transfer to the environment conditioned on correct visual grounding. On VirtualHome and ALFWorld, open 4B–27B models exceed 90% success in both environments, and our smallest agent substantially outperforms a 27B direct visual policy in each. Constraints and search prove complementary rather than interchangeable: in ALFWorld either alone solves under a third of tasks, whereas their combination solves over 95%. The method also uses several times fewer generated tokens than extended thinking and far fewer model-visible images than direct interaction, and residual failures localize to state acquisition rather than plan generation without any specialized training.

1 Introduction

Vision-language models (VLMs) provide a appealing interface for embodied agents. They recognize objects, draw on commonsense knowledge about environments, and generate plausible action sequences from natural-language goals. These capabilities suggest a direct route from perception to control, i.e., show the model what the agent sees, request the next action, execute it, and repeat. In the context of long-horizon tasks, however, plausibility is likely no sufficient. A single action applied to the wrong instance, before its preconditions hold, or under an incorrect belief about the scene can invalidate the remaining trajectory. Such errors accumulate over interaction, and neither model scale nor fluent reasoning guarantees that the final sequence is executable (Valmeekam et al., 2023; Kambhampati et al., 2024; Valmeekam et al., 2025; He et al., 2025, *inter alia*).

This limitation reflects a difficult coupling of perception and planning. From an egocentric image, an agent must identify task-relevant objects, determine their spatial relations and states, preserve these facts across viewpoints, select applicable actions, and reason about their delayed effects. A direct policy must solve all of these inside one autoregressive history, where they are free to compound: a fluent plan may reference an unobserved object, and a correct observation may still be followed by an inapplicable action. Execution feedback can expose such an error after it occurs, but does not prevent the model from repeating an invalid choice or drifting from the state its own actions induced. Existing embodied

*Equal contribution.

systems mitigate parts of this through affordance models, replanning, or task-and-motion planning (Ahn et al., 2022; Huang et al., 2023; Yang et al., 2024). Reliable high-level task planning nonetheless still requires a mechanism that binds visually acquired state to formal action semantics during generation itself.

In this paper, we present a neurosymbolic embodied agent for discrete, long-horizon household tasks that supplies this mechanism by factoring the problem into two connected phases (see Figure 1 for a quick overview). *Phase I: Exploration* couples a VLM to an exploration harness that acquires only the objects, relations, and instance bindings the goal requires. Model priors guide where to inspect, while multiple viewpoints and grounded interactions determine which relational claims enter the symbolic state. *Phase II: Exploitation* plans from this grounded initial state under a PDDL transition model. At every decoding step, the model distribution is restricted to tokens that can extend an applicable action, and Monte Carlo tree search (MCTS) evaluates executable continuations with a domain-independent planning heuristic, combining the language model’s procedural prior with explicit state transitions.

The decomposition separates validity from goal reachability. State-dependent constrained decoding prevents the search from generating inapplicable actions, while MCTS distinguishes among the remaining executable plans by their long-horizon consequences. Applicability thereby becomes an invariant maintained throughout generation and search rather than a behavior the model must re-infer at each turn, and every returned plan is executable under the transition model by construction. The guarantee transfers to the simulator when the symbolic model is sound and the visually acquired instance binding is correct. This qualification is important: formal constraints cannot recover an object that Phase I fails to perceive. They instead localize the residual uncertainty to state acquisition, rather than allowing perceptual, grounding, and planning errors to compound throughout an unconstrained trajectory. Our failure attribution confirms that this is where the error in fact concentrates.

We evaluate on VirtualHome and ALFWorld, two visually grounded household benchmarks with different action semantics and grounding demands. Across Qwen3.5 models from 4B to 27B parameters, the agent obtains 94.5–99.5% success in VirtualHome and 90.3–97.8% in ALFWorld. The 4B agent exceeds the 27B direct visual policy by 32.0 percentage points in VirtualHome and 63.4 points in ALFWorld, indicating that explicit structure can substitute for considerable model scale. It is also efficient: with costs paired by model size and averaged across benchmarks, the method uses up to $4.0\times$ fewer generated tokens than extended thinking, $1.5\times$ fewer than unconstrained MCTS, and $5.6\times$ fewer model-visible images than direct interaction.

Our primary contributions are: a) a two-phase architecture that separates task-directed visual state acquisition from goal-directed symbolic planning, through a grounded initial-state interface; b) a state-dependent decoding procedure that enforces PDDL action applicability at token level, coupled with language-guided MCTS over executable plan prefixes, so that plans are executable by construction; and c) an empirical end-to-end evaluation across two embodied household benchmarks, including direct visual and embodied-model baselines, component ablations, inference and observation costs, and causal failure attribution.

2 Related Work

Our work connects symbolic task planning, language-model planning, constrained decoding, and embodied perception to generate executable plans from visual observations.

Symbolic Planning and LLM Planners Symbolic task planning searches over explicit world states. PDDL is the dominant modeling language (McDermott et al., 1998; Fox and Long, 2003) where grounding predicates and operators yields a task whose actions are executable only when their preconditions hold, after which their effects update the state. This enables exact feasibility checks, although the combinatorial grounded action space motivates heuristic planners such as Fast Downward (Helmert, 2006). We use the same applicability semantics online, to constrain LLM decoding and MCTS. LLMs supply useful procedural priors but do not reliably enforce these semantics and it has been shown that fluency does not imply correct reasoning over preconditions, effects, and state change (Valmeekam et al., 2023, *inter alia*), and even frontier models require external validation and exhibit hallucinated actions or unstable plans when problems are expressed in language (Kambhampati et al., 2024; Corrêa, Pereira, and Seipp, 2026; Armony, Meroño-Peñuela, and Canal, 2025; Katz et al., 2026).

LLMs with PDDL and Symbolic Planners LLM+P and NL2Plan translate language into PDDL and invoke a planner, incrementally constructing domains and problems from minimal descriptions (Liu et al., 2023; Gestrin, Kuhlmann, and Seipp, 2024), but generated domains can be syntactically plausible yet semantically incorrect (Oswald et al., 2024; Smirnov et al., 2024; Zhang et al., 2024). Later systems iteratively refine PDDL with agents and verifiers, expose symbolic transitions as tools, or induce action models from demonstrations (La Malfa et al., 2025; Göbel et al., 2026; Huang et al., 2025). These approaches largely treat PDDL as a generated representation or downstream solver input.

We instead expose grounded operator applicability *during* inference, rejecting invalid continuations and restricting tree search to executable prefixes.

Constrained Decoding for Structured Generation Constrained decoding masks next tokens that violate a formal specification, supporting structured generation without fine-tuning and with efficient, reliable handling of subword tokenization, schema evaluation, and dynamic agent outputs (Geng et al., 2023; Beurer-Kellner, Fischer, and Vechev, 2024; Ugare et al., 2024; Dong et al., 2024; Geng et al., 2025; Park, Zhou, and D’Antoni, 2025; Li et al., 2026). Most of this work addresses syntax or text-only semantic control. Closest to ours, SEM-CTRL couples controlled decoding with search under an external transition model to enforce semantic validity, and later work learns such constraints automatically under an oracle (Albinhassan, Madhyastha, and Russo, 2026; Albinhassan et al., 2025). We extend state-dependent constraints to visually grounded planning, where the state the constraints act on is not given but must first be acquired through perception and interaction.

Perception and Planning Separating representation construction from goal-directed control is a long-standing design pattern in robotics: SLAM estimates a reusable environment model for subsequent navigation (Cadena et al., 2016), while modular agents combine learned mapping, semantic priors over likely goal locations, and online scene graphs with analytic planning and exploration-objective selection (Chaplot et al., 2020b,a; Yokoyama et al., 2024; Yin et al., 2025). The shared division of labor is that semantic knowledge guides where to acquire information, whereas grounded observations determine the represented state. Language-conditioned systems apply the same split, pairing LLM proposals with affordances, tools, or execution feedback (Ahn et al., 2022; Yao et al., 2023; Huang et al., 2023), and VLM-TAMP uses a VLM to propose horizon-reducing subgoals while a task-and-motion planner enforces geometric and kinematic feasibility (Yang et al., 2024). A systematic study of multimodal planning reaches a complementary conclusion where we see that VLMs perform substantially better as PDDL formalizers than as end-to-end long-horizon planners, with incomplete visual extraction of object relations the principal bottleneck (He et al., 2025). This directly motivates our decomposition of visual state acquisition from constrained symbolic planning.

3 Method: Neurosymbolic Embodied Agents

We consider long-horizon embodied task planning at the level of discrete, semantically meaningful actions, rather than continuous control or motion generation. These tasks couple two distinct challenges: a) an agent must first determine what is present and where it is, then b) synthesize a sequence of actions that achieves the goal. A model must resolve occlusion, containment, and entity grounding before reasoning over action preconditions and long-horizon effects. Solving both in one autoregressive loop entangles perceptual and planning errors where a fluent plan may reference an unobserved object, while a correct observation may still be followed by an inapplicable action. We factor this tightly coupled problem into two explicitly connected subproblems. During *Phase I: Exploration*, a vision-language model (VLM) actively acquires the scene facts needed by the task. During *Phase II: Exploitation*, a language model plans from the resulting symbolic representation while a formal transition model constrains every action. The interface assigns state acquisition to Phase I and valid, goal-directed deliberation to Phase II.

Problem setting. We model each task as a partially observable Markov decision process (POMDP) $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$ (Kaelbling, Littman, and Cassandra, 1998). Here, $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces; $T(s' | s, a)$ is the transition model; Ω is the space of egocentric visual observations; and $O(I | s)$ is the observation model. The sparse reward R is positive only when the resulting state satisfies goal formula G . We consider a finite, undiscounted horizon ($\gamma = 1$).

At time t , the agent receives only an egocentric RGB observation $I_t \in \Omega$, while the complete environment state $s_t \in \mathcal{S}$ remains latent, we note that objects may be outside the field of view, occluded, or enclosed. The output is a plan $\pi = (a_1, \dots, a_n)$ whose actions are sequentially applicable and whose final state satisfies the goal, written $s_n \models G$. We do not solve the POMDP in belief space and emphasize that Phase I acts to reduce uncertainty over the task-relevant state and commits to a point estimate, after which Phase II solves a deterministic classical planning problem and returns an open-loop plan. This reduction is exact when the dynamics reachable from that estimate are deterministic and the estimate is correct on the literals the plan depends on conditions we make precise in the validity guarantee below. Generative models provide useful plan priors but do not ensure accurate state estimation or applicability. We present a higher-level overview in Figure 1.

Goal specifications. The goal formula G is obtained by a deterministic, rule-based parse of each benchmark’s own symbolic goal annotation, applied uniformly to every episode (we explain more in Experimental Setup). We note that this is not hand-specified and has only class-specific information (hence no potential for leakage of information).

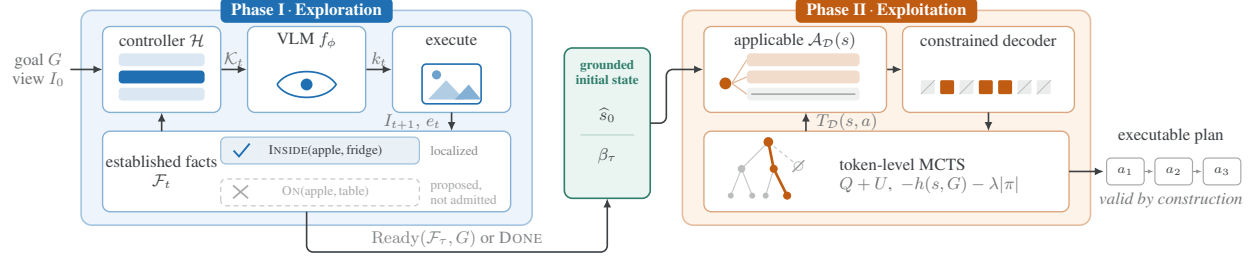


Figure 1: **Two phases joined by a narrow symbolic interface.** A literal enters $\mathcal{F}_t$ only under visual or interaction evidence, so priors decide *where to look*, not *what is present* (greyed chip: unsupported). Only $\hat{s}_0$ and β_τ cross into Phase II, which excludes at three scales: inapplicable actions (struck pill), illegal tokens (slashed cells), unexpanded branches (struck node).

3.1 Phase I (Exploration): Task-Directed Visual Grounding

To avoid exhaustive scene reconstruction, a VLM, denoted by f_ϕ with parameters ϕ , is coupled to a deterministic exploration controller $\mathcal{H}$ (harness). The VLM selects inspection targets from egocentric frames, while the harness tracks inspection history, grounds skills, and enforces environment interaction constraints. This is shown in the left half of Figure 1. We now formalize the exploration phase below.

Let $\mathcal{L}$ be the set of possible grounded literals, such as $\text{INSIDE}(\text{apple}, \text{fridge})$. After t exploration steps, the established facts form $\mathcal{F}_t \subseteq \mathcal{L}$; $\mathcal{R}_t$ records inspected locations, attempted skills, and outcomes. The controller function $\mathcal{H}$ constructs candidate skills

$$\begin{aligned} \mathcal{K}_t &= \mathcal{H}(G, \mathcal{F}_t, \mathcal{R}_t) \\ &\subseteq \{\text{GOTO}(x), \text{OPEN}(x), \text{CLOSE}(x), \text{LOOK}, \text{DONE}\}. \end{aligned} \quad (1)$$

Here x denotes a navigable object, surface, or receptacle. The skill family is fixed, while $\mathcal{K}_t$ is a state-dependent subset because targets that have already been inspected or are not yet grounded may be omitted. LOOK is an active inspection macro: it acquires additional viewpoints and, when possible, invokes a grounded interaction to test an object–receptacle hypothesis. Such interactions, including temporary pickup and restoration, are internal grounding operations rather than VLM-selected task actions. DONE terminates exploration. The harness may require another inspection or container-opening action when evidence is insufficient, but it does not predict unseen contents.

Given the goal, current image, established facts, interaction record, and candidate skills, it proposes

$$(\mathcal{P}_t, k_t) = f_\phi(G, I_t, \mathcal{F}_t, \mathcal{R}_t, \mathcal{K}_t), \quad k_t \in \mathcal{K}_t. \quad (2)$$

Here $\mathcal{P}_t$ is the set of facts proposed from the current image, such as $\text{VISIBLE}(o)$, $\text{ON}(o, r)$, or $\text{INSIDE}(o, r)$, and k_t is the selected exploration skill. The symbols o and r denote an object and a supporting surface or receptacle, respectively.

The environment grounds and executes k_t , returning image I_{t+1} and event e_t . The event records success or failure and, on failure, a concise grounding error. The evidence update is

$$(\mathcal{F}_{t+1}, \mathcal{R}_{t+1}, \beta_{t+1}) = \mathcal{U}(\mathcal{F}_t, \mathcal{R}_t, \beta_t, \mathcal{P}_t, k_t, I_{t+1}, e_t), \quad (3)$$

where β_t maps symbolic names to environment instances and $\mathcal{U}$ is a deterministic evidence-update function. It adds action effects only after successful execution and commits object–receptacle bindings only when supported by localization from multiple viewpoints or by a successful grounded interaction. Unsupported or conflicting proposals may guide another inspection but do not establish a relational fact. This is not a general hallucination detector, so visual recognition errors can still cause Phase I failure.

Let τ denote the final exploration step. Exploration terminates when $\text{Ready}(\mathcal{F}_\tau, G)$ is true, when the model selects DONE, or when the interaction budget is exhausted. The Boolean function Ready tests whether the objects and relations needed to instantiate the planning problem have been grounded. Phase I returns the grounded initial state

$$\hat{s}_0 = \mathcal{F}_\tau \cup \mathcal{F}_{\text{str}}, \quad \beta_\tau, \quad (4)$$

where $\mathcal{F}_{\text{str}}$ contains fixed ontology and domain facts shared across tasks, and β_τ is the final symbol-to-instance binding. Neither quantity contains privileged scene state.

3.2 Phase II (Exploitation): Constrained Symbolic Planning

As illustrated in the right half of Figure 1, Phase II combines grounded state $\hat{s}_0$ with a PDDL transition model $\mathcal{D}$, formulating classical planning task II. In simple terms, the language model supplies a procedural prior over plans, while

PDDL enforces action applicability and tree search evaluates long-horizon consequences. This retains the model’s semantic preference without requiring it to infer the transition dynamics from the prompt.

Formally, the problem $\Pi = \langle \mathcal{D}, \hat{s}_0, G \rangle$ comprises domain $\mathcal{D}$, grounded initial state $\hat{s}_0$, and goal G . A grounded operator a has preconditions $\text{pre}(a)$ and effects $\text{add}(a)$ and $\text{del}(a)$. For state s , the applicable actions and successor are

$$\mathcal{A}_{\mathcal{D}}(s) = \{a \mid \text{pre}(a) \subseteq s\}, \quad (5)$$

$$T_{\mathcal{D}}(s, a) = (s \setminus \text{del}(a)) \cup \text{add}(a), \quad a \in \mathcal{A}_{\mathcal{D}}(s). \quad (6)$$

Thus $\mathcal{A}_{\mathcal{D}}(s)$ contains actions whose preconditions hold, and $T_{\mathcal{D}}$ computes their successors. Applicability is available during generation, not only after sampling a plan.

State-dependent constrained decoding. Constraints enforce PDDL applicability at token decoding boundaries. Let $\mathcal{V}$ be the token vocabulary and $\text{tok}(a)$ the tokens spelling action a . At state s , let $\mathcal{L}_{\mathcal{D}}(s) = \{\text{tok}(a) \mid a \in \mathcal{A}_{\mathcal{D}}(s)\}$ be the applicable action strings, and let $\text{Prefix}(\cdot)$ return all their prefixes. For partial action u , the allowed next tokens are

$$\mathcal{C}_{\mathcal{D}}(s, u) = \{v \in \mathcal{V} \mid u \cdot v \in \text{Prefix}(\mathcal{L}_{\mathcal{D}}(s))\}, \quad (7)$$

where $u \cdot v$ appends token v to prefix u . Let c_{Π} denote the fixed model context for planning problem Π , including the task goal, grounded initial state, and action interface. The language model with parameters θ assigns next-token probability $p_{\theta}(v \mid u, c_{\Pi})$. Its state-constrained distribution is

$$p_{\theta, \mathcal{C}}(v \mid s, u, c_{\Pi}) \propto p_{\theta}(v \mid u, c_{\Pi}) \mathbb{I}[v \in \mathcal{C}_{\mathcal{D}}(s, u)], \quad (8)$$

where $\mathbb{I}[\cdot]$ is one when its condition holds and zero otherwise. Thus invalid tokens receive no probability; the remaining model mass is renormalized implicitly. At each action boundary, $T_{\mathcal{D}}$ advances the state and the constraint is recomputed. Unlike a surface-form grammar, the distribution is state dependent: the same action can be permitted in one state and excluded in another.

Validity guarantee. Every complete plan generated under distribution $p_{\theta, \mathcal{C}}$ is applicable under transition model $\mathcal{D}$ by construction. By induction, if a prefix reaches state s_i , Equation 7 restricts the subsequent action to $\mathcal{A}_{\mathcal{D}}(s_i)$, and Equation 6 advances the state to s_{i+1} . This validity guarantee transfers to the environment provided domain model $\mathcal{D}$ is sound, and Phase I binding β_{τ} is accurate.

Constrained language-guided search. An applicable sequence may still fail to reach G as validity and goal correctness are distinct (see Albinhassan, Madhyastha, and Russo, 2026, for a more detailed discussion). We optimize the latter with token-level MCTS. A node $n = (s, u)$ contains state s and partial action u . Selection is restricted to $\mathcal{C}_{\mathcal{D}}(s, u)$ and combines the backed-up search value with the language-model prior:

$$v^* = \arg \max_{v \in \mathcal{C}_{\mathcal{D}}(s, u)} [Q(n, v) + U(n, v)], \quad (9)$$

$$U(n, v) = c_N p_{\theta, \mathcal{C}}(v \mid s, u, c_{\Pi}) \frac{\sqrt{N(n)}}{1 + N(n, v)}.$$

Here $n = (s, u)$ is a search node containing symbolic state s and current token prefix u , while v is a candidate next token. $Q(n, v)$ is the value estimate, $p_{\theta, \mathcal{C}}(v \mid s, u, c_{\Pi})$ is the constrained model prior, $N(n)$ and $N(n, v)$ are parent and edge visit counts, and c_N controls exploration. The second term is a policy-prior upper-confidence bound (PUCB) (Silver et al., 2017), a prior-weighted variant of UCT (Kocsis and Szepesvári, 2006). It lets the model order plausible continuations while search corrects locally likely choices with poor downstream effects.

Completed action sequences (or plans (π)) are evaluated using domain-independent planning heuristic:

$$V(\pi) = \begin{cases} R_G, & s_{\pi} \models G, \\ -h(s_{\pi}, G) - \lambda|\pi|, & \text{otherwise.} \end{cases} \quad (10)$$

where, s_{π} is the state reached by π , $R_G > 0$ rewards the goal, $h(s_{\pi}, G) \geq 0$ estimates remaining symbolic distance, and $\lambda > 0$ penalizes plan length. Standard planning heuristics instantiate h , so no learned critic is required. Search stops at a goal or returns the highest-valued valid candidate within budget. If the finite search tree contains a solution at token depth d , the constraints preserve every applicable action, and each permitted token has positive prior, PUCB visits the solution path with probability approaching one as the simulation budget grows; this is an asymptotic completeness statement, not a finite-budget guarantee. PDDL supplies feasibility, the language model supplies semantic preference, and MCTS supplies global deliberation over their intersection.

4 Experimental Setup

4.1 Benchmarks

We evaluate in two complementary household simulators whose scenes span kitchens, living rooms, bedrooms, and bathrooms. **VirtualHome** represents activities as executable programs over objects in furnished Unity apartments (Puig et al., 2018). The dataset consists of 200 unique problems, testing novel object instances across four goal families: placing objects in a dishwasher, microwave, or refrigerator, and preparing food. These tasks primarily require multi-object rearrangement and correct reasoning about containment, support, proximity, and container state. **ALFWorld** aligns abstract household reasoning with visually embodied execution in AI2-THOR (Shridhar et al., 2021). We evaluate all 134 episodes in its unseen split. The episodes cover six task families: pick-and-place, placing two objects, examining an object under a light, and cleaning, heating, or cooling an object before placement. Compared with VirtualHome, ALFWorld places greater emphasis on egocentric search, instance-level grounding, closed receptacles, and irreversible state transformations. Agents receive RGB observations and the task goal; simulator scene graphs and admissible-action lists are never exposed. For both benchmarks, a fixed rule-based parser converts the benchmark-provided symbolic goal annotation into G using only goal classes and relations, not instance identities. It is applied uniformly, and the VLM has no access to the scene state, instance IDs, trajectories, or solutions.

4.2 Models and Baselines

Our main experiments use the Qwen3.5 family at 4B, 9B, and 27B parameters, allowing controlled comparison of model scale. We compare against direct interactive VLM policies, which predict one action from the current egocentric frame, and against Phase II ablations that isolate reasoning, search, and formal constraints: unconstrained greedy decoding, unconstrained thinking, constrained greedy decoding, unconstrained MCTS, and constrained MCTS. All Phase II comparisons reuse the same model-specific grounded initial state, so they differ only in planning. We additionally evaluate the open embodied models Embodied-R1.5 (Yuan et al., 2026), MiMo-Embodied-7B (Hao et al., 2025), and RoboBrain2.0-32B (BAAI RoboBrain Team et al., 2025), together with API-based Gemini-3-Flash, as direct visual policies.

All methods receive the task goal, the same egocentric RGB interface, and the same simulator-generated outcome channel: success after a grounded action, or a concise error when the action is invalid or unreachable. Direct policies choose one action per turn. Our exploration harness additionally tracks inspection history and curates high-level exploration skills; this controller is part of the evaluated method and does not expose privileged scene state to the VLM. Phase I decodes greedily and terminates when the task-relevant objects and relations are grounded or after N_{exp} steps. Interaction limits are 20 steps in VirtualHome and 55 in ALFWorld. Greedy conditions use temperature zero; thinking conditions use the model-recommended sampling parameters and a 32K-token context.

MCTS evaluates nonterminal prefixes with domain-independent planning heuristics. We use a weighted average of common heuristics such as Fast Forward, LM-cut, and remaining plan cost by Downward (Helmert, 2006; Hoffmann and Nebel, 2001; Helmert and Domshlak, 2009). Additional details are presented in Appendix A.

4.3 Metrics

The primary metric is task success, which is one only when the goal state is reached as defined by the environment’s simulator. We also report generated tokens, model-visible images, and failure attribution, and use successful plan length to characterize benchmark difficulty. Generated-token cost sums output tokens across all model calls for an episode. Image cost counts RGB frames provided to the model, rather than frames rendered internally by the simulator. Failure attribution assigns each unsuccessful episode to the first causal stage that prevents task completion.

5 Results

We consider four core research questions a) whether factorizing visual perception and planning improves task completion over direct visual policies and unconstrained reasoning models; b) how inference-time search and token-level constraints interact to drive plan validity and goal reachability; c) whether these performance gains reduce generated-token and environment-observation costs; and finally d) how residual failures are causally distributed across visual state acquisition, action grounding, and search. We present the headline results here and note that we provide additional results in Appendix C.

Approach	Model	VH	ALFW
<i>Neurosymbolic embodied agent</i>			
Ours	Qwen3.5-4B	94.5	90.3
Ours	Qwen3.5-9B	99.5	97.8
Ours	Qwen3.5-27B	99.0	95.5
<i>Direct Qwen policies</i>			
Direct VLM	Qwen3.5-4B	5.5	1.5
Direct VLM	Qwen3.5-9B	10.0	6.0
Direct VLM	Qwen3.5-27B	62.5	26.9
<i>API policy</i>			
Direct VLM	Gemini-3-Flash	91.0	50.8
<i>Embodied VLM policies</i>			
Embodied VLM	MiMo-Embodied-7B	18.5	0.7
Embodied VLM	Embodied-R1.5-8B	1.0	6.0
Embodied VLM	RoboBrain2-32B	27.0	9.7
<i>Planning from the Phase-I representation</i>			
Reasoning Enabled	Qwen3.5-4B	47.0	5.6
Reasoning Enabled	Qwen3.5-9B	31.5	10.8
Reasoning Enabled	Qwen3.5-27B	59.0	16.9

Table 1: Task success (%) on VirtualHome and ALFWorld. Direct VLM and embodied VLM policies predict one environment action per interaction step. Best results are bold.

5.1 Overall Task Success

Table 1 reports end-to-end task success. The proposed method reaches 94.5–99.5% in VirtualHome and 90.3–97.8% in ALFWorld, substantially outperforming direct visual interaction and unconstrained chain-of-thought reasoning² across all model scales. These results show that perception and long-horizon planning, which typically compound errors when entangled in a single autoregressive policy, now become tractable when factorized into task-directed state acquisition and executable symbolic search.

The high success rates are consistent with this division as Phase I asks the VLM to recover only a small set of task-relevant facts from targeted observations, producing a grounded PDDL problem file (s_0, β_τ) . Phase II then solves an explicit symbolic problem rather than maintaining visual state and action semantics throughout a long interaction. The resulting agent is neurosymbolic in a functional sense. The VLM contributes visual recognition and commonsense priors, the transition model defines admissible state change, and MCTS deliberates over the intersection. None of these components alone must learn the complete embodied policy.

The factorization is also markedly parameter efficient. Our 4B agent obtains 94.5% and 90.3% success rates, exceeding the 27B direct VLM by 32.0% and 63.4% points in VirtualHome and ALFWorld, respectively. It likewise exceeds unconstrained thinking with the 27B model by 35.5 and 73.4 points. Thus, we see that increasing model capacity or allowing additional free-form reasoning does not substitute for an executable action space. A small model operating over a grounded transition system is more reliable than a substantially larger model required to infer perception, state tracking, action semantics, and long-horizon control jointly.

Gemini-3-Flash is the strongest direct-policy baseline, achieving 91.0% in VirtualHome and 50.8% in ALFWorld. We also note that the embodied-specialized baselines show limited transfer in this zero-shot setting. MiMo-Embodied-7B, Embodied-R1.5-8B, and RoboBrain2-32B achieve low success rates. In our setup, we are evaluating the zero-shot control through previously unseen simulator action interfaces under strict end-state goal verification. For example, MiMo-Embodied is trained across planning, affordance, and spatial tasks, Embodied-R1.5 introduces its own Planner–Grounder–Corrector framework, and RoboBrain2 targets spatial understanding and temporal decision-making (Hao et al., 2025; Yuan et al., 2026; BAAI RoboBrain Team et al., 2025). This result highlights the gap between pre-trained embodied priors and maintaining an executable, stateful policy over dozens of interaction turns (see He et al., 2025, for more details). We furnish additional results in Appendix C.

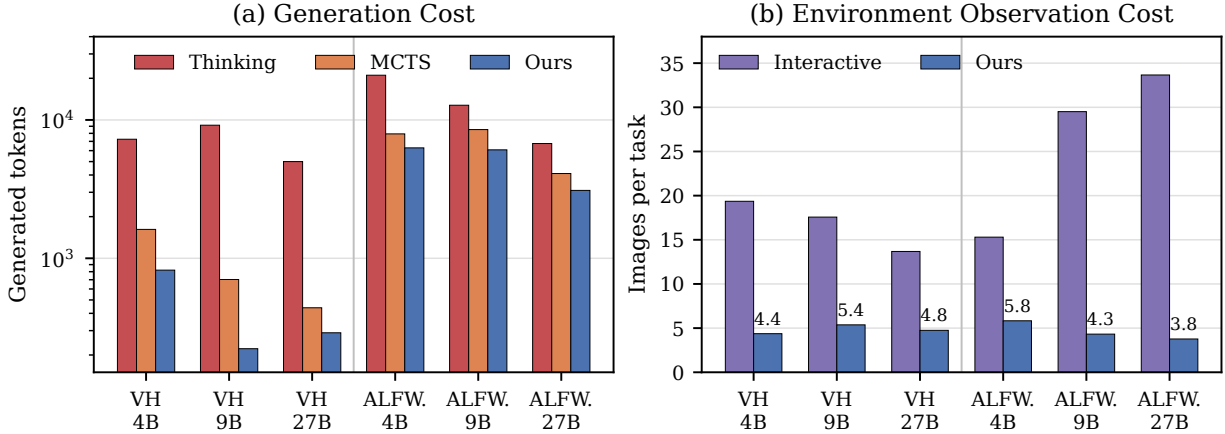


Figure 2: Inference and environment-observation cost across model scales. (a) Mean generated tokens per task for unconstrained thinking, unconstrained MCTS, and our constrained MCTS. (b) Mean model-visible images per task for direct interactive policies and our two-phase agent. Lower is better in both panels.

5.2 Inference and Environment Cost

Figure 2 shows that reliability does not necessitate higher inference or environment interaction costs. We compare methods at matched model scale and average their costs across the two benchmarks. Under this paired comparison, constrained MCTS uses up to $1.5\times$ fewer generated tokens than unconstrained MCTS and $4.0\times$ fewer than extended thinking. Constraints remove inapplicable branches before the model spends tokens expanding them; search is then concentrated on executable alternatives rather than used as repeated generate-and-reject sampling.

The reduction in visual environment queries is even more pronounced. Using the same fixed-scale averaging setup, our agent requires up to $5.5\times$ fewer model-visible images than direct interaction, typically four to six rather than 13–34. Phase I obtains a compact task-relevant representation once, after which Phase II compares complete candidate plans without repeatedly querying the visual environment. This amortizes perception across the search tree. Visual inputs incur encoder computation that is not reflected in generated-token counts, so token-only comparisons understate the saving. In physical deployment, fewer perception–decision cycles also imply fewer environment queries and less unnecessary exploratory motion, although image count is not identical to low-level motor-action count.

5.3 Failure Analysis

We assign every unsuccessful episode to the earliest causal boundary that prevents completion. *Exploration/representation* denotes an insufficient or incorrect Phase I state, such as a missed goal object or an erroneous object–receptacle binding. *Grounding/semantics* applies to direct policies whose proposed action cannot be executed with the intended effect because a precondition fails, an instance is incorrect or unreachable, or the symbolic action cannot be mapped to the simulator. *Goal not reached* applies when execution remains possible but the goal is false at termination, including search- or interaction-budget exhaustion, loops, and premature DONE.

We present our results in Figure 3. We observe that the methods fail at different boundaries. Our residual error is concentrated upstream where we see that about 2.3% of VirtualHome tasks and 4.5% of ALFWorld tasks fail during representation acquisition, while only 1.0% of ALFWorld tasks acquire a usable state but do not reach the goal. Once the symbolic state is accurate, constrained planning is therefore highly reliable. Qualitative inspection shows a characteristic visual failure where small, low-contrast objects, such as a fork blending into a tabletop, may remain unrecognized unless viewed at close range. The failure then propagates because the grounded initial state omits a required object. This is a limitation of the visual state-acquisition component rather than the applicability guarantee. We expect improvements in visual recognition and active viewpoint selection should transfer directly to the complete method.

Direct Qwen policies instead fail to reach the goal on 64.2% of evaluated VirtualHome tasks. In ALFWorld, 39.8% fail at grounding and a further 48.8% terminate without satisfying the goal. The shift toward grounding in ALFWorld is consistent with its denser instance and receptacle semantics, where selecting a plausible object class is insufficient without binding the correct instance and interaction.

²frequently referred to as thinking mode

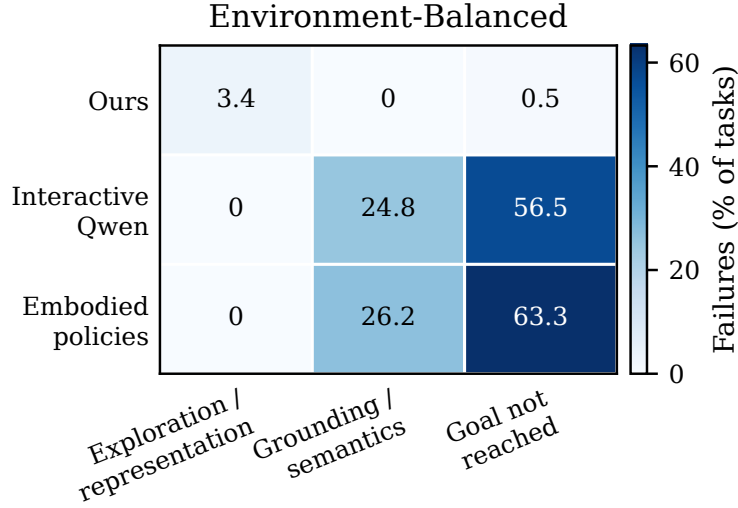


Figure 3: Terminal failure attribution, macro-averaged equally across VirtualHome and ALFWorld. Values are percentages of all tasks.

Embodied policies show the same qualitative bottleneck where failures concentrate in goal completion in VirtualHome and are divided between grounding and goal completion in ALFWorld. Table 1 confirms that our proposal embeds action validity as an invariant within the constrained decoder and transition model, relieving the language model from inferring preconditions at each turn. As a result, failure modes isolate to state acquisition, with remaining errors stemming from incorrect visual representations rather than invalid plan generation. Additional results are in Appendices C and D.

5.4 Constrained Planning Ablations

Method	Search	Constraints	VH	ALFW
Greedy	–	–	61.0	15.4
Thinking	–	–	59.0	16.9
Greedy	–	✓	98.5	32.3
MCTS	✓	–	99.0	29.2
Ours	✓	✓	99.0	95.5

Table 2: Component ablation with Qwen3.5-72B. Entries report end-to-end task success (%).

Table 2 shows that neither additional generation nor either component in isolation explains the final result. In VirtualHome, constrained greedy decoding raises success from 61.0% to 98.5%, showing that applicability constraints account for most of the gain; MCTS without constraints reaches 99.0%, but at the higher generation cost shown in Figure 2. ALFWorld is more discriminating: constraints alone reach 32.3% and unconstrained MCTS 29.2%, whereas their combination reaches 95.5%. The larger gap is consistent with a harder planning problem: successful 72B plans average 8.0 high-level actions in ALFWorld and 6.4 in VirtualHome, while the respective maxima are 30 and eight; ALFWorld also introduces more instance and receptacle bindings at each stage. Search and constraints are therefore complementary rather than interchangeable. Constraints make every explored prefix executable, while MCTS corrects locally plausible choices whose downstream consequences do not achieve the goal. Their nonlinear gain in ALFWorld supports the neurosymbolic design: symbolic semantics control feasibility, and model-guided search supplies goal-directed preference over the remaining valid plans.

6 Conclusion

We presented a neurosymbolic agent for long-horizon embodied task planning that separates task-directed visual exploration from constrained symbolic planning: a VLM and exploration harness construct a grounded initial state, while state-dependent decoding and MCTS generate a goal-directed plan under explicit PDDL dynamics. This factorization converts action applicability from a behavior the model must repeatedly infer into a constraint maintained

throughout generation and search, enabling open 4B–27B models to substantially outperform larger direct visual policies, unconstrained reasoning, and embodied-specialized baselines while using fewer generated tokens and visual observations.

References

- Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Fu, C.; Gopalakrishnan, K.; Hausman, K.; Herzog, A.; Ho, D.; Hsu, J.; Ibarz, J.; Ichter, B.; Irpan, A.; Jang, E.; Jauregui Ruano, R.; Jeffrey, K.; Jesmonth, S.; Joshi, N. J.; Julian, R.; Kalashnikov, D.; Kuang, Y.; Lee, K.-H.; Levine, S.; Lu, Y.; Luu, L.; Parada, C.; Pastor, P.; Quiambao, J.; Rao, K.; Rettinghouse, J.; Reyes, D.; Sermanet, P.; Sievers, N.; Tan, C.; Toshev, A.; Vanhoucke, V.; Xia, F.; Xiao, T.; Xu, P.; Xu, S.; Yan, M.; and Zeng, A. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv:2204.01691*.
- Albinhassan, M.; Madhyastha, P.; Law, M.; and Russo, A. 2025. Learning and Enforcing Context-Sensitive Control for LLMs. In Zhao, J.; Wang, M.; and Liu, Z., eds., *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, 834–842. Vienna, Austria: Association for Computational Linguistics. ISBN 979-8-89176-254-1.
- Albinhassan, M.; Madhyastha, P.; and Russo, A. 2026. SEM-CTRL: Semantically Controlled Decoding. *Transactions on Machine Learning Research*.
- Armony, M.; Meroño-Peñuela, A.; and Canal, G. 2025. How Far Are LLMs from Symbolic Planners? An NLP-Based Perspective. *arXiv:2508.01300*.
- BAAI RoboBrain Team; Cao, M.; Tan, H.; Ji, Y.; Lin, M.; Li, Z.; et al. 2025. RoboBrain 2.0 Technical Report. *arXiv preprint arXiv:2507.02029*.
- Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2024. Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation. In Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; and Berkenkamp, F., eds., *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 3658–3673. PMLR.
- Cadena, C.; Carlone, L.; Carrillo, H.; Latif, Y.; Scaramuzza, D.; Neira, J.; Reid, I.; and Leonard, J. J. 2016. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics*, 32(6): 1309–1332.
- Chaplot, D. S.; Gandhi, D.; Gupta, A.; and Salakhutdinov, R. 2020a. Object Goal Navigation Using Goal-Oriented Semantic Exploration. In *Advances in Neural Information Processing Systems*, volume 33.
- Chaplot, D. S.; Gandhi, D.; Gupta, S.; Gupta, A.; and Salakhutdinov, R. 2020b. Learning To Explore Using Active Neural SLAM. In *International Conference on Learning Representations (ICLR)*.
- Corrêa, A. B.; Pereira, A. G.; and Seipp, J. 2026. Frontier Large Language Models Rival State-of-the-Art Planners. *arXiv:2511.09378*.
- Dong, Y.; Ruan, C. F.; Cai, Y.; Lai, R.; Xu, Z.; Zhao, Y.; and Chen, T. 2024. Xgrammar: Flexible and efficient structured generation engine for large language models. *Proceedings of Machine Learning and Systems 7*.
- Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.
- Geng, S.; Cooper, H.; Moskal, M.; Jenkins, S.; Berman, J.; Ranchin, N.; West, R.; Horvitz, E.; and Nori, H. 2025. Generating Structured Outputs from Language Models: Benchmark and Studies. *arXiv:2501.10868*.
- Geng, S.; Josifoski, M.; Peyrard, M.; and West, R. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 10932–10952. Singapore: Association for Computational Linguistics.
- Gestrin, E.; Kuhlmann, M.; and Seipp, J. 2024. Towards Robust LLM-Driven Planning from Minimal Text Descriptions. In *ICAPS 2024 Workshop on Human-Aware Explainable Planning*.
- Göbel, K.; Lorang, P.; Zips, P.; and Glück, T. 2026. Agentic LLM Planning via Step-Wise PDDL Simulation: An Empirical Characterisation. *arXiv:2603.06064*.
- Hao, X.; Zhou, L.; Huang, Z.; Hou, Z.; Tang, Y.; et al. 2025. MiMo-Embodied: X-Embodied Foundation Model Technical Report. *arXiv preprint arXiv:2511.16518*.
- He, M.; Zheng, Y.; Liu, Y.; An, Z.; Cai, B.; Huang, J.; Zhou, L.; Liu, F.; Li, Z.; and Zhang, L. 2025. Vision Language Models Cannot Plan, but Can They Formalize? *arXiv preprint arXiv:2509.21576*.
- Helmert, M. 2006. The Fast Downward Planning System. *Journal of Artificial Intelligence Research*, 26: 191–246.

- Helmert, M.; and Domshlak, C. 2009. Landmarks, Critical Paths and Abstractions: What’s the Difference Anyway? In *International Conference on Automated Planning and Scheduling*, volume 19, 162–169.
- Hoffmann, J.; and Nebel, B. 2001. The FF Planning System: Fast Plan Generation Through Heuristic Search. *Journal of Artificial Intelligence Research*, 14: 253–302.
- Huang, J.; Xiao, Y.; Zhang, Z.; Coates, M.; HAO, J.; and Zhang, Y. 2025. One Demo Is All It Takes: Planning Domain Derivation with LLMs from A Single Demonstration. In *Workshop on Foundation Models Meet Embodied Agents at CVPR 2025*.
- Huang, W.; Xia, F.; Xiao, T.; Chan, H.; Liang, J.; Florence, P.; Zeng, A.; Tompson, J.; Mordatch, I.; Chebotar, Y.; Sermanet, P.; Jackson, T.; Brown, N.; Luu, L.; Levine, S.; Hausman, K.; and Ichter, B. 2023. Inner Monologue: Embodied Reasoning through Planning with Language Models. In Liu, K.; Kulic, D.; and Ichnowski, J., eds., *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, 1769–1782. PMLR.
- Kaelbling, L. P.; Littman, M. L.; and Cassandra, A. R. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 101(1–2): 99–134.
- Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L. P.; and B Murthy, A. 2024. Position: LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks. In Salakhutdinov, R.; Kolter, Z.; Heller, K.; Weller, A.; Oliver, N.; Scarlett, J.; and Berkenkamp, F., eds., *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 22895–22907. PMLR.
- Katz, M.; Kokel, H.; Srinivas, K.; and Sohrabi, S. 2026. Planning in the LLM Era: Building for Reliability and Efficiency. arXiv:2605.21902.
- Kocsis, L.; and Szepesvári, C. 2006. Bandit Based Monte-Carlo Planning. In *European Conference on Machine Learning*, 282–293.
- La Malfa, E.; Zhu, P.; Marro, S.; Bernardini, S.; and Wooldridge, M. 2025. End-to-End PDDL Planning with Hardcoded and Dynamic Agents. arXiv:2512.09629.
- Li, L.; Dong, Y.; Wang, G.; Xu, Z.; Jiang, A.; and Chen, T. 2026. XGrammar-2: Dynamic and Efficient Structured Generation Engine for Agentic LLMs. In *Proceedings of the ACM Conference on AI and Agentic Systems*, 1009–1022. New York, NY, USA: Association for Computing Machinery. ISBN 9798400724152.
- Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; and Stone, P. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477.
- McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL: The Planning Domain Definition Language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.
- Oswald, J.; Srinivas, K.; Kokel, H.; Lee, J.; Katz, M.; and Sohrabi, S. 2024. Large Language Models as Planning Domain Generators. In *34th International Conference on Automated Planning and Scheduling*.
- Park, K.; Zhou, T.; and D’Antoni, L. 2025. Flexible and Efficient Grammar-Constrained Decoding. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 48262–48275. PMLR.
- Puig, X.; Ra, K.; Boben, M.; Li, J.; Wang, T.; Fidler, S.; and Torralba, A. 2018. VirtualHome: Simulating Household Activities via Programs. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8494–8502.
- Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*.
- Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; et al. 2017. Mastering the Game of Go without Human Knowledge. *Nature*, 550(7676): 354–359.
- Smirnov, P.; Joubin, F.; Ceravola, A.; and Gienger, M. 2024. Generating consistent PDDL domains with Large Language Models. arXiv:2404.07751.
- Ugare, S.; Suresh, T.; Kang, H.; Misailovic, S.; and Singh, G. 2024. SynCode: LLM Generation with Grammar Augmentation. arXiv:2403.01632.
- Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; and Kambhampati, S. 2023. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36: 38975–38987.
- Valmeekam, K.; Stechly, K.; Gundawar, A.; and Kambhampati, S. 2025. A Systematic Evaluation of the Planning and Scheduling Abilities of the Reasoning Model o1. *Transactions on Machine Learning Research*.

- Yang, Z.; Garrett, C.; Fox, D.; Lozano-Pérez, T.; and Kaelbling, L. P. 2024. Guiding Long-Horizon Task and Motion Planning with Vision Language Models. *arXiv:2410.02193*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.
- Yin, H.; Xu, X.; Zhao, L.; Wang, Z.; Zhou, J.; and Lu, J. 2025. UniGoal: Towards Universal Zero-shot Goal-oriented Navigation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
- Yokoyama, N.; Ha, S.; Batra, D.; Wang, J.; and Bucher, B. 2024. VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation. In *International Conference on Robotics and Automation (ICRA)*.
- Yuan, Y.; Huang, Y.; Yao, X.; Li, Y.; Zhang, S.; et al. 2026. Embodied-R1.5: Evolving Physical Intelligence via Embodied Foundation Models. *arXiv preprint arXiv:2606.11324*.
- Zhang, T.; Zhang, L.; Hou, Z.; Wang, Z.; Gu, Y.; Clark, P.; Callison-Burch, C.; and Tandon, N. 2024. PROC2PDDL: Open-Domain Planning Representations from Texts. In Dalvi Mishra, B.; Durrett, G.; Jansen, P.; Lipkin, B.; Neves Ribeiro, D.; Wong, L.; Ye, X.; and Zhao, W., eds., *Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations (@ACL 2024)*, 13–24. Bangkok, Thailand: Association for Computational Linguistics.

A Extended Experimental Setup

A.1 Evaluation Protocol

We evaluate 200 VirtualHome tasks and all 134 episodes in the ALFWorld unseen split. Each agent receives the natural-language goal and egocentric RGB observations. The simulator scene graphs, object states, instance identifiers, and admissible actions are not exposed to the model. Environment adapters ground emitted class-level actions to simulator instances and return only whether the action executed successfully (e.g., PREVIOUS ACTION EXECUTED SUCCESSFULLY) or a concise failure reason when it is invalid or unreachable (e.g., YOUR ACTION COULD NOT BE GROUNDED FROM THIS POSE). All methods and baselines receive the same environment feedback. Note that, in our method and planning-from-Phase I representation ablations, only Phase I receives environment feedback while Phase II does not. In comparison, direct-policy and visual-interaction baselines receive feedback at every environment step. Task success is determined exclusively from the environment simulation (e.g., Unity for VirtualHome).

Our method separates visual state acquisition from planning. Phase I uses greedy decoding and a deterministic exploration harness to acquire goal-relevant predicates. It stops when the relevant state is grounded or after N_{exp} steps. Phase II receives the resulting symbolic handoff without any visual inputs and produces an open-loop plan of at most N_{plan} actions, without access to the environment executor. $N_{exp} + N_{plan}$ is restricted to 20 in VirtualHome and 55 in ALFWorld.

This appendix documents the evaluation protocol and model hyperparameters used for all experiments.

A.2 Direct Visual Interaction

Direct policies emit one class-level action per turn from the current first-person observation. The environment parser accepts only actions in the benchmark-specific DSL, grounds the referenced class to an executable instance, advances the simulator after successful actions, and renders the next observation. Successful actions receive only a success acknowledgment, and failed actions receive a short grounding or reachability error and an unchanged observation. This is the same feedback Phase I receives. The model must infer action postconditions and what action to execute next from the task description, feedback, updated image, and interaction history. Direct policies terminate on simulator success, an emitted DONE, or the interaction limit: 20 steps in VirtualHome and 55 in ALFWorld. The interaction limit here is matched to $N_{exp} + N_{plan}$ in our method to ensure fairness. Direct Qwen, API, and embodied VLM policies all follow this same protocol.

A.3 Sampling Parameters

Table 3 gives the decoding settings, following the usage recommended by the model providers. Phase I, direct Qwen, greedy planning, constrained greedy planning, and both MCTS conditions use greedy decoding. The main-paper reasoning-enabled rows likewise report temperature-zero decoding. In this appendix, we further evaluate Qwen thinking using its recommended sampling settings over seeds 0, 42, and 1738. MiMo and Embodied-R1.5 use their recommended sampling configurations over three seeds (the main paper reports the seed-0 run). RoboBrain’s main result uses its

recommended greedy configuration, and we provide additional sensitivity analysis using nucleus sampling over three seeds here.

Condition	Thinking	Temperature	Top- p	Top- k	Max output tokens
Qwen Phase I exploration	off	0.0	1.0	–	512
Qwen direct interactive	off	0.0	1.0	–	128 per turn
Qwen greedy Phase II	off	0.0	1.0	–	512
Qwen MCTS Phase II	off	0.0	1.0	–	128 per expansion
Qwen thinking, main paper	on	0.0	1.0	–	32,768
Qwen thinking, supplementary	on	1.0	0.95	20	32,768
MiMo-Embodied-7B	off	0.6	0.95	–	512 per turn
Embodied-R1.5-8B	off	0.7	0.8	20	512 per turn
RoboBrain2.0-32B, main paper	off	0.0	1.0	–	512 per turn
RoboBrain2.0-32B, supplementary	off	1.0	0.9	–	512 per turn

Table 3: Decoding settings. A dash denotes no top- k cutoff. Qwen supplementary thinking, MiMo, Embodied-R1.5, and RoboBrain nucleus sampling are evaluated with seeds 0, 42, and 1738.

A.4 MCTS

Both MCTS variants use a token-level prior-weighted form of UCT with expansion width 16, exploration constant 2.5, base constant 10, and early termination on a PDDL-valid goal plan. We restrict MCTS to 250 seconds per task. Hyperparameters were selected empirically. Constrained MCTS masks tokens that cannot extend an applicable PDDL action and caches reusable constrained prefixes. Unconstrained MCTS searches the full model distribution. Candidates are evaluated using a fixed weighted combination of domain-independent Fast Downward heuristics, including Fast Forward, LM-cut, and remaining plan cost. The same search settings are used for all models and benchmarks.

A.5 Cluster Specifications

Experiments used two GPU clusters with NVIDIA A100-80GB and NVIDIA H200 GPUs. Each evaluation job used one GPU, 8–16 CPU cores, and 128 GB RAM. Local-model inference used bfloat16 precision with vLLM.

Benchmark	Per episode	One run	Three runs
VirtualHome	\$0.0148	\$2.97	\$8.90
ALFWorld	\$0.0534	\$7.16	\$21.48
Combined	–	\$10.13	\$30.38

Table 4: Estimated Gemini 3 Flash high-thinking API cost in USD.

Table 4 shows the API costs for the Gemini evaluation runs. Local inference avoids per-token API charges, and our method is also more accurate.

B Statistical Analysis

B.1 Multiple Runs

We quantify stochastic sensitivity with three repeated runs; local-model runs use seeds 0, 42, and 1738. For each condition, we report the mean success rate, sample standard deviation across runs, and a 95% percentile confidence interval from 10,000 task-cluster bootstrap replicates. Each replicate resamples task indices and applies the same indices to all three runs, preserving the dependence among repeated evaluations of the same task. Deterministic conditions are evaluated once.

Table 5 reports deterministic and stochastic conditions in a common format. The deterministic rows have zero decoding variance. Recommended-sampling Qwen thinking remains substantially below our method at every scale and on both benchmarks. Nucleus sampling does not improve RoboBrain’s VirtualHome result and remains weak in ALFWorld. Among local conditions using neither constraints nor search, Qwen3.5-27B thinking is strongest, reaching 57.5% in VirtualHome and 18.5% in ALFWorld. The main paper reports a single Gemini 3 Flash run. Here, we additionally rerun Gemini 3 Flash three times with high thinking, reaching, on average, 87.2% in VirtualHome and 55.0% in ALFWorld.

Approach	Model	VH		ALFW	
		Accuracy (%)	95% CI	Accuracy (%)	95% CI
Ours	Qwen3.5-4B	94.5 $\pm$ 0.0	[91.0, 97.5]	90.3 $\pm$ 0.0	[85.1, 94.8]
Ours	Qwen3.5-9B	99.5 $\pm$ 0.0	[98.5, 100.0]	97.8 $\pm$ 0.0	[94.8, 100.0]
Ours	Qwen3.5-27B	99.0 $\pm$ 0.0	[97.5, 100.0]	95.5 $\pm$ 0.0	[91.8, 98.5]
Direct VLM	Qwen3.5-4B	5.5 $\pm$ 0.0	[2.5, 9.0]	1.5 $\pm$ 0.0	[0.0, 3.7]
Direct VLM	Qwen3.5-9B	10.0 $\pm$ 0.0	[6.0, 14.5]	6.0 $\pm$ 0.0	[2.2, 10.0]
Direct VLM	Qwen3.5-27B	62.5 $\pm$ 0.0	[55.5, 69.0]	26.9 $\pm$ 0.0	[19.4, 34.3]
Qwen thinking	Qwen3.5-4B	39.7 $\pm$ 3.1	[34.2, 45.3]	5.3 $\pm$ 0.5	[2.4, 8.8]
Qwen thinking	Qwen3.5-9B	31.0 $\pm$ 9.1	[25.5, 36.8]	14.1 $\pm$ 0.4	[8.7, 20.0]
Qwen thinking	Qwen3.5-27B	57.5 $\pm$ 7.9	[51.5, 63.3]	18.5 $\pm$ 0.8	[12.3, 25.1]
Embodied VLM	MiMo-Embodied-7B	13.3 $\pm$ 4.5	[10.5, 16.5]	0.5 $\pm$ 0.4	[0.0, 1.5]
Embodied VLM	Embodied-R1.5-8B	0.3 $\pm$ 0.6	[0.0, 0.8]	4.0 $\pm$ 1.9	[1.5, 7.0]
Embodied VLM	RoboBrain2.0-32B	27.0 $\pm$ 0.0	[21.0, 33.0]	9.7 $\pm$ 0.0	[5.2, 14.9]
RoboBrain nucleus	RoboBrain2.0-32B	0.0 $\pm$ 0.0	[0.0, 0.0]	3.5 $\pm$ 1.1	[1.7, 5.5]
API VLM (high thinking)	Gemini 3 Flash	87.2 $\pm$ 0.8	[83.7, 90.3]	55.0 $\pm$ 1.6	[47.3, 62.4]

Table 5: Accuracy and robustness across model conditions. The value after $\pm$ is the sample standard deviation across three runs. Deterministic conditions are marked ± 0.0 . Intervals are 95% percentile task-cluster bootstrap intervals with repeated-run outcomes held together within each resample. Bold denotes the highest accuracy in each benchmark and results not significantly different from it.

Gemini uses 11.0 images per VirtualHome episode on average across the three runs, increasing to 23.2 images in ALFWorld. By comparison, the 27B method uses 4.8 and 3.8 model-visible images in Phase I, after which Phase II plans without further image-conditioned model calls. The factorized method therefore requires fewer image-conditioned interactions, and the final plan is executed in a single open-loop pass.

B.2 Paired Significance Testing

We use exact paired McNemar tests for task-level success. When one condition is stochastic, it is paired against the deterministic condition separately for each run, and we retain the largest p -value, yielding a conservative test across runs. Holm correction controls the family-wise error rate at $\alpha = 0.05$. We define two hypothesis families: (1) main comparisons, covering our method against direct policies, unconstrained thinking, embodied and API policies, and model-scale comparisons, and (2) component ablations, covering the 27B comparisons against constrained greedy and unconstrained MCTS. Holm correction is applied separately within each family.

Comparison	VH		ALFW	
	Holm p	Sig.	Holm p	Sig.
Ours (Qwen3.5-4B) vs. Direct VLM (Qwen3.5-4B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-9B) vs. Direct VLM (Qwen3.5-9B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-27B) vs. Direct VLM (Qwen3.5-27B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-4B) vs. Qwen thinking (Qwen3.5-4B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-9B) vs. Qwen thinking (Qwen3.5-9B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-27B) vs. Qwen thinking (Qwen3.5-27B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-4B) vs. Direct VLM (Qwen3.5-27B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-4B) vs. Qwen thinking (Qwen3.5-27B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-9B) vs. Embodied VLM (RoboBrain2.0-32B)	< 0.001	✓	< 0.001	✓
Ours (Qwen3.5-4B) vs. API VLM (high thinking) (Gemini 3 Flash)	0.123	–	< 0.001	✓
Ours (Qwen3.5-4B) vs. Ours (Qwen3.5-9B)	0.025	✓	0.010	✓
Ours (Qwen3.5-9B) vs. Ours (Qwen3.5-27B)	1.000	–	0.500	–

Table 6: Exact paired McNemar tests for the main comparisons ($m = 24$ tests), with Holm correction within the family. For stochastic conditions, correction is applied to the maximum exact p -value across runs.

As shown in Table 6, after correction, our method significantly outperforms the matched direct Qwen and recommended-sampling thinking baselines at every model scale in both environments. The 4B method also significantly exceeds the 27B direct and thinking baselines. Against Gemini 3 Flash with high thinking, the 4B method is not significantly

Comparison	VirtualHome		ALFWorld	
	Holm p	Sig.	Holm p	Sig.
Ours vs. Constrained greedy	1.000	–	< 0.001	✓
Ours vs. Unconstrained MCTS	1.000	–	< 0.001	✓

Table 7: Exact paired McNemar tests for the component ablations ($m = 4$ tests). All conditions use Qwen3.5-27B. Holm correction is applied within the family.

different after Holm correction in VirtualHome and is significantly stronger in ALFWorld. Scaling our method from 4B to 9B is significant in both environments after correction, while the difference between 9B and 27B is not significant in either.

In the ablations (see Table 7), constrained greedy and unconstrained MCTS are not significantly different from our method after Holm correction in VirtualHome; both are significantly weaker in ALFWorld, where combining search with transition-model constraints provides the decisive gain. The paired tests substantiate the paper’s central task-accuracy claims.

C Additional Results

C.1 Failure Attribution by Model and Environment

Figure 4 resolves terminal failures by environment and by model, including the embodied policies and the Gemini API baseline, providing a finer-grained breakdown than the grouped results presented in the main text.

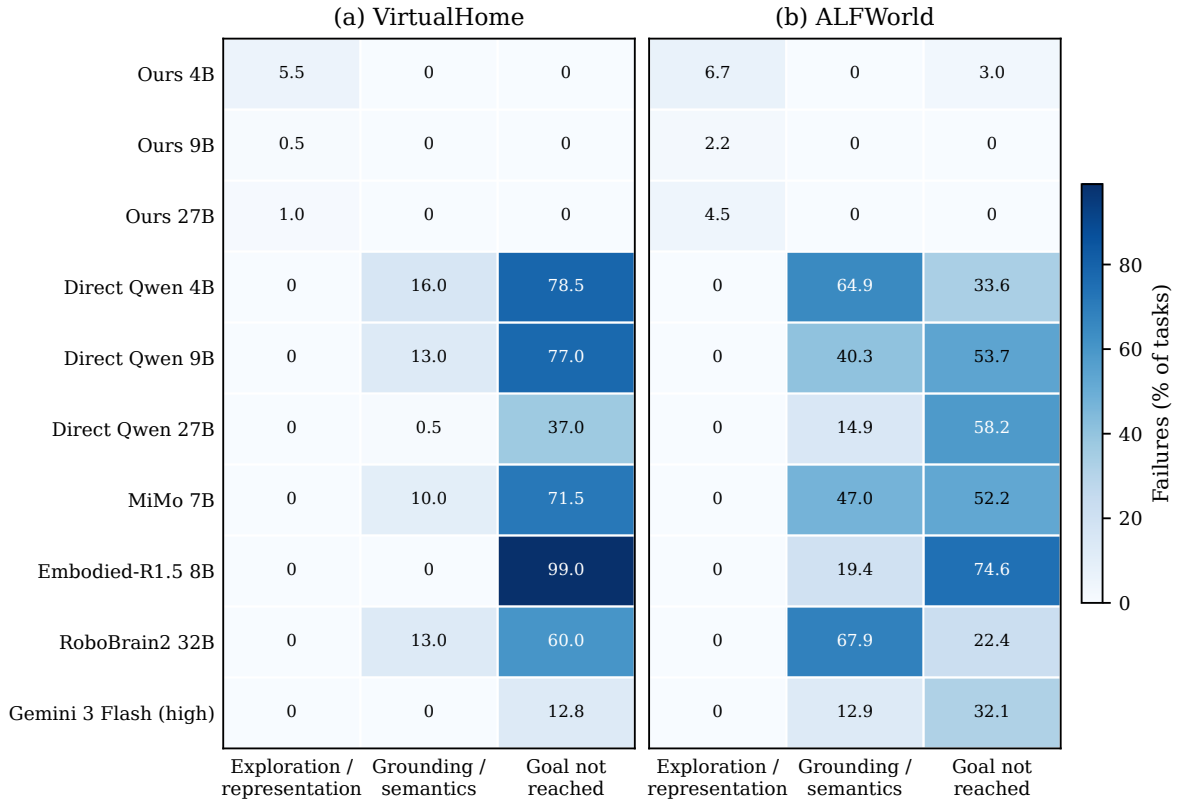


Figure 4: Terminal failure attribution without aggregation across environments, Qwen scales, or visual-policy models. Values are percentages of all tasks for the corresponding row. Successful tasks contribute to none of the failure categories.

Discussion. The disaggregated results expose two trends hidden by the main-paper average. First, increasing the direct Qwen model scale reduces terminal grounding failures: from 16.0% to 0.5% in VirtualHome and from 64.9% to 14.9% in ALFWorld. However, the 27B policy still terminates without reaching the goal on 37.0% and 58.2% of tasks, respectively. Scale therefore improves action grounding more than long-horizon completion. Second, the embodied models fail at different boundaries despite similarly low overall accuracy. Embodied-R1.5 predominantly emits executable trajectories that do not complete the goal, whereas MiMo and RoboBrain encounter substantially more grounding failures in ALFWorld. Across Gemini’s three high-thinking runs, 12.8% of VirtualHome evaluations reach the interaction limit. In ALFWorld, 12.9% terminate after repeated grounding failures, and 32.1% terminate without reaching the goal. Our method’s failures remain concentrated in representation acquisition from Phase I, and only the ALFWorld 4B introduces a search-budget failure.

C.2 Phase-II Results across Model Scales

The main paper reports the 27B component ablation. Table 8 gives task success for every Qwen scale and Phase-II condition. The supplementary thinking row uses the recommended sampling configuration and reports the mean across three seeds, and the temperature-zero row is the single-run condition reported in the main paper.

Condition	VirtualHome			ALFWorld		
	4B	9B	27B	4B	9B	27B
Unconstrained greedy	66.0 $\pm$ 0.0	59.0 $\pm$ 0.0	61.0 $\pm$ 0.0	2.4 $\pm$ 0.0	10.0 $\pm$ 0.0	15.4 $\pm$ 0.0
Thinking, temperature zero	47.0 $\pm$ 0.0	31.5 $\pm$ 0.0	59.0 $\pm$ 0.0	5.6 $\pm$ 0.0	10.8 $\pm$ 0.0	16.9 $\pm$ 0.0
Thinking, recommended sampling	39.7 $\pm$ 3.1	31.0 $\pm$ 9.1	57.5 $\pm$ 7.9	5.3 $\pm$ 0.5	14.1 $\pm$ 0.4	18.5 $\pm$ 0.8
Constrained greedy	84.0 $\pm$ 0.0	99.5 $\pm$ 0.0	98.5 $\pm$ 0.0	24.0 $\pm$ 0.0	31.5 $\pm$ 0.0	32.3 $\pm$ 0.0
Unconstrained MCTS	78.0 $\pm$ 0.0	93.0 $\pm$ 0.0	99.0 $\pm$ 0.0	4.8 $\pm$ 0.0	22.3 $\pm$ 0.0	29.2 $\pm$ 0.0
Ours	94.5 $\pm$ 0.0	99.5 $\pm$ 0.0	99.0 $\pm$ 0.0	90.3 $\pm$ 0.0	97.8 $\pm$ 0.0	95.5 $\pm$ 0.0

Table 8: Task success (%) for every Qwen3.5 model scale and Phase-II condition, reported as accuracy $\pm$ standard deviation. Recommended-sampling thinking values use three seeds, and all other conditions use deterministic decoding. Bold denotes the best result in each column and any result not significantly different from it.

Discussion. The full scale breakdown reinforces the main ablation. Constraints alone are highly effective in VirtualHome, particularly for 9B and 27B, but neither constraints nor unconstrained search closes the ALFWorld gap. Their combination raises ALFWorld success to 90.3–97.8% across all three scales. Recommended sampling changes the unconstrained-thinking result but does not alter this ordering.

D Fine-Grained Failure Analysis

The preceding attribution separates failures by causal stage. We further inspect the terminal mechanism recorded by the interaction harness and audit representative traces. Table 9 reports mutually exclusive terminal outcomes for the direct and embodied policies. These labels describe how an episode ended, rather than automatically inferring visual causality.

The finer categories clarify why increased scale does not eliminate direct-policy failures. For direct Qwen in VirtualHome, terminal grounding failures fall from 16.0% at 4B to 0.5% at 27B, while interaction-limit failures remain at 37.0%. In ALFWorld, grounding failures similarly fall from 64.9% to 14.9%, but the 27B model still loses 44.8% of all tasks to the interaction limit and another 13.4% to premature model termination. Larger models therefore produce more executable actions without consistently converting those actions into completed long-horizon goals.

The embodied policies exhibit distinct recovery behaviors. Embodied-R1.5 reaches the interaction limit or emits DONE on 99.0% of VirtualHome tasks, whereas RoboBrain’s dominant ALFWorld outcome is repeated ungroundable action generation (67.9%). MiMo lies between these two, with both grounding and budget exhaustion contributing substantially in ALFWorld. The aggregate embodied-policy row in the main paper consequently combines qualitatively different limitations.

D.1 Response to Interaction Failure

Terminal categories do not reveal whether a policy changes strategy after an unsuccessful action. We therefore measure behavior following the first ungroundable or failed execution. Table 10 reports the frequency of such events, eventual task recovery, immediate repetition of the failed class-level request, and persistent action loops.

Environment	Policy	Model	Grounding	Interaction limit	Model DONE
VirtualHome	Direct Qwen	4B	16.0	71.5	7.0
VirtualHome	Direct Qwen	9B	13.0	77.0	0.0
VirtualHome	Direct Qwen	27B	0.5	37.0	0.0
VirtualHome	Embodied	MiMo-7B	10.0	40.0	31.5
VirtualHome	Embodied	R1.5-8B	0.0	50.0	49.0
VirtualHome	Embodied	RoboBrain-32B	13.0	60.0	0.0
ALFWorld	Direct Qwen	4B	64.9	32.1	1.5
ALFWorld	Direct Qwen	9B	40.3	35.1	18.7
ALFWorld	Direct Qwen	27B	14.9	44.8	13.4
ALFWorld	Embodied	MiMo-7B	47.0	35.8	16.4
ALFWorld	Embodied	R1.5-8B	19.4	68.7	6.0
ALFWorld	Embodied	RoboBrain-32B	67.9	17.2	5.2

Table 9: Fine-grained terminal failure mechanisms as percentages of all tasks for each designated main-paper run. Grounding counts terminal mechanisms only. A policy can encounter many failed actions (Table 10) without terminating on the ungroundable threshold.

Environment	Policy	Model	Encounters failure	Recovers task	Repeats request	Persistent loop
VirtualHome	Direct Qwen	4B	64.5	5.4	74.5	45.5
VirtualHome	Direct Qwen	9B	53.5	9.3	33.0	79.5
VirtualHome	Direct Qwen	27B	29.0	37.9	8.3	9.5
VirtualHome	Embodied	MiMo-7B	95.0	17.4	59.6	43.5
VirtualHome	Embodied	R1.5-8B	72.5	0.7	4.6	58.0
VirtualHome	Embodied	RoboBrain-32B	93.0	22.0	62.1	50.0
ALFWorld	Direct Qwen	4B	73.9	1.0	71.8	94.8
ALFWorld	Direct Qwen	9B	79.9	5.6	41.7	72.4
ALFWorld	Direct Qwen	27B	75.4	25.7	18.4	42.5
ALFWorld	Embodied	MiMo-7B	96.3	0.0	63.3	87.3
ALFWorld	Embodied	R1.5-8B	70.9	5.3	52.1	92.5
ALFWorld	Embodied	RoboBrain-32B	93.3	7.2	84.3	85.1

Table 10: Failure response of direct and embodied policies (%). Encounters failure is the fraction of tasks containing an ungroundable or failed execution. Recovers task is success among those affected tasks. Repeats request is the fraction of failed-action transitions followed by the same class-level request. Persistent loop is the fraction of all tasks containing at least five identical consecutive requests or a six-step two-action oscillation.

Scaling direct Qwen substantially improves local recovery. In VirtualHome, the fraction of failed-action transitions followed by the same request falls from 74.5% at 4B to 8.3% at 27B, and among episodes that encounter an execution failure, eventual success rises from 5.4% to 37.9%. ALFWorld shows the same direction, although it remains harder: Qwen-27B recovers 25.7% of affected episodes and still enters a persistent loop on 42.5% of all tasks. The non-monotonic VirtualHome loop rate at 9B reflects a distinction between termination and persistence: weaker policies may exhaust the grounding-failure threshold early, whereas an executable but unproductive policy can continue until the larger step budget is exhausted.

The embodied models fail differently rather than uniformly. In VirtualHome, R1.5 rarely repeats the exact failed request (4.6%) but frequently enters alternating-view or repeated-action loops (58.0%), matching the rotation sequence in Figure 5. In ALFWorld, RoboBrain repeats the same class-level request after 84.3% of failed transitions, while MiMo encounters at least one failed execution in 96.3% of tasks and never recovers one of those affected episodes in the seed-0 run. Explicit negative feedback is therefore available but is not consistently converted into a revised grounded strategy.

D.2 Representation-Level Errors

An audit of VirtualHome method failures across model scales identifies several failures involving small objects (e.g., pie, juice), which are compact food objects whose class and supporting relation must be recovered from a wide household view. The model may identify a room and supporting surface without resolving the small goal object, or identify both entities without committing their support relation. In ALFWorld, long searches can instead visit the appropriate region of the scene and inspect many plausible receptacles without exposing the target before the

exploration budget expires. Thus, the residual Phase-I errors separate into *local perceptual ambiguity* and *incomplete search coverage*, neither of which is captured by a single missing-object count.

D.3 Representative Policy Traces

Figure 5 illustrates four failure mechanisms with example episodes from each simulator and task family. All prevalence statements come from the complete recorded results in Table 9.

Together, these examples separate four mechanisms that success rate alone conflates: local perception, finite active-search coverage, repeated state reversal, and view-control oscillation after an interaction failure. They also explain why scale alone is insufficient. A larger model may select a semantically appropriate receptacle or action more reliably, while still lacking the persistent scene memory, global progress state, or recovery policy required to complete the episode. The method moves much of that burden into an explicit handoff and planner, but remains bounded by what Phase I can expose within its exploration budget. Direct and embodied policies avoid a fixed symbolic bottleneck, yet must repeatedly solve grounding and progress tracking online.

E Prompts

All prompts use the same templates across model scales. The examples below instantiate those templates for a VirtualHome task that places a wineglass in the dishwasher. ALFWorld uses similarly styled prompts. Figures 6–8 show the instantiated prompts for each phase.

E.1 Phase I: Task-Directed Exploration

At each exploration step, the VLM receives a first-person view, the goal-relevant object classes, the accumulated belief, prior grounded actions, environment feedback, and a harness-curated set of currently available high-level inspection skills. A representative prompt is shown in Figure 6.

The complete template additionally defines the accepted predicate schema, benchmark class-name normalization, and the semantics of actions. The model may select only one of the high-level skills supplied for the current state. The harness grounds that skill, executes it, and accepts relational facts only when supported by the resulting observation or interaction.

E.2 Phase II: Planning

Phase II receives the class-level action reference, the grounded state produced by Phase I, and the symbolic goal. Instance identifiers remain hidden. No image is passed to Phase II. A representative unconstrained prompt is shown in Figure 7.

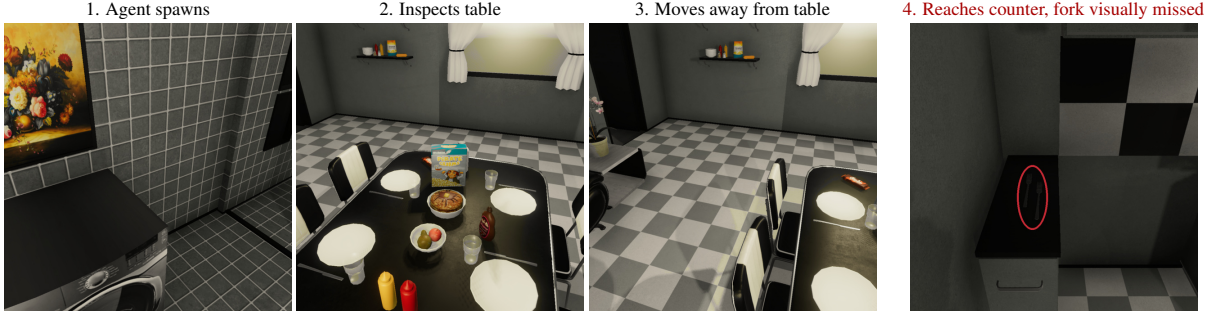
The constrained conditions use the same natural-language prompt. Their only difference is at decoding time: after every completed action, the PDDL transition model updates the symbolic state and masks token continuations that cannot form an applicable next action.

E.3 Direct Visual Interaction

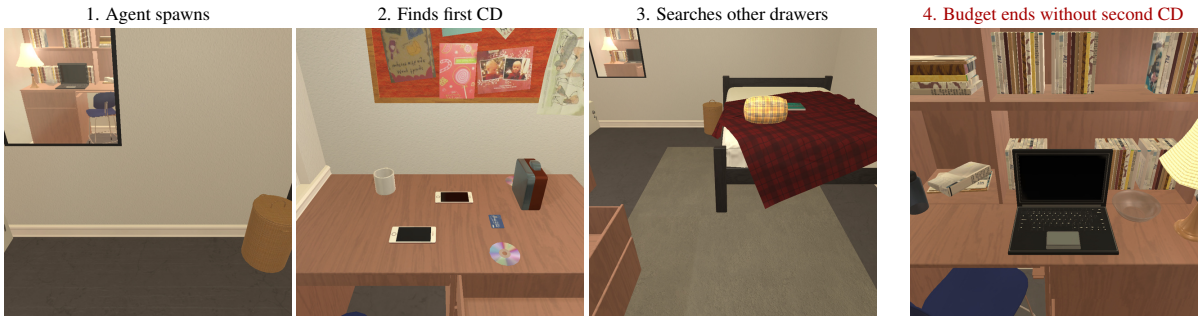
The direct Qwen policy begins with a prompt describing first-person observation, reachability, the action DSL, class-name conventions, and the requirement to emit exactly one action. Figure 8 shows the first user turn, action vocabulary, environment feedback, and the history representation.

This feedback exposes neither a valid-action list nor a symbolic state. Successful feedback deliberately omits inferred postconditions, since the model must interpret the updated observation and reason from the action semantics. Embodied policies receive the same task and interaction setup, shown in Figure 8.

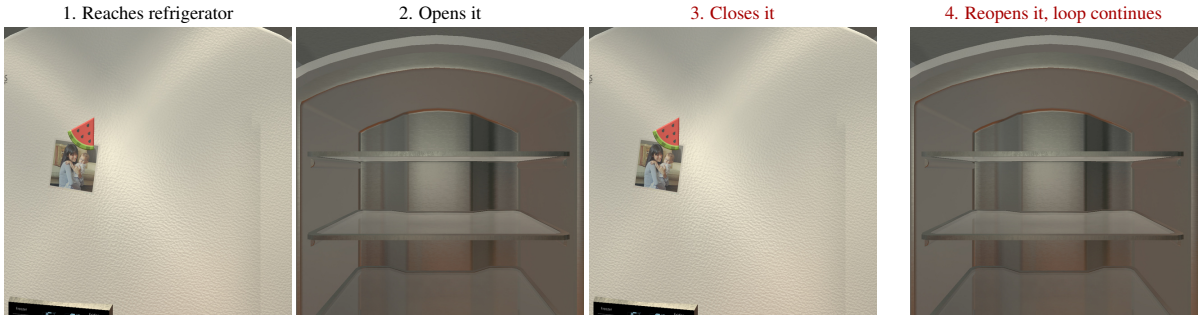
MiMo and RoboBrain return a bare action. Embodied-R1.5 returns the same action inside `<answer>...</answer>`. Thinking is disabled for all three embodied-specialized policies.



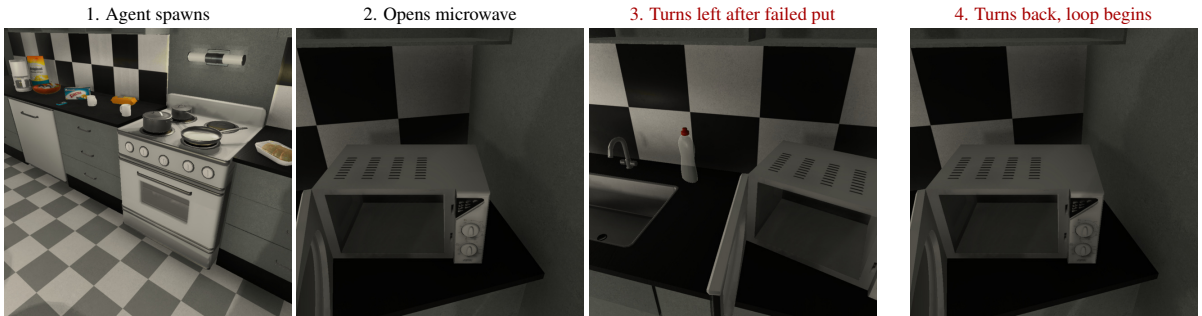
(a) **Method, VirtualHome:** directed exploration reaches the correct counter, but the visually subtle fork is not recovered.



(b) **Method, ALFWorld:** one required CD is found, but active search exhausts its budget before localizing the second instance.



(c) **Direct Qwen, ALFWorld:** valid actions repeatedly reverse the same refrigerator state until the episode ends.



(d) **Embodied-R1.5, VirtualHome:** after reaching the appliance, manipulation failure devolves into view oscillation.

Figure 5: Example failure sequences. Each row follows an episode from its initial or first informative state through progress to the decisive error and its terminal consequence. The red callout in (a) identifies the relevant object.

You are a vision-language agent in VirtualHome, a household simulator. You see one or more first-person RGB images. Your only job in this phase is to find the listed objects and determine where each one is. Do not plan or execute the household chore. Reply with JSON only.

Task:
Find each listed object from first-person vision and determine where it is.

Objects to find and locate:
wineglass, dishwasher

Current belief:
visible kitchen; visible kitchentable; visible dishwasher; closed dishwasher

Previous actions:
1. goto kitchen -> success
2. goto kitchentable -> success

Valid skills right now:
- goto wineglass
- look

Feedback:
The previous action executed successfully.
Return visible predicates, optional object locations, exactly one valid skill in "action", and "done": true only after every listed object has a grounded placement.

Figure 6: Example VirtualHome Phase-I exploration prompt.

You control an embodied agent in a simulated household called VirtualHome. Output a sequence of actions that changes the observable initial state into the goal state. An action takes effect only if all of its conditions already hold. The agent cannot act on an object it is not close to.

Actions:
walk <object/room>: move close to an object or room
grab <object>: hold a nearby object
open <container>: open a nearby closed container
putin <object> <container>: place a held object in a nearby open container
DONE

Initial state:
visible kitchen
visible kitchentable
visible dishwasher
on wineglass kitchentable
closed dishwasher
handempty

Goal:
inside wineglass dishwasher

Each action should make progress toward the goal and respect preconditions. Output one action per line using class names only, and stop immediately after the action that achieves the goal.

Plan:

Figure 7: Example VirtualHome Phase-II planning prompt.

FIRST USER TURN

Goal: Put the wineglass in the dishwasher.

Goal objects (exact VirtualHome class names): wineglass, dishwasher

Use the first-person image and choose the next action toward the goal.

Output only one action line (or DONE).

ACTION FORMS

walk <object/room>

grab <object>

open <container>

close <container>

putback <object> <surface>

putin <object> <container>

turnleft; turnright; walkforward; DONE

SUCCESSFUL-ACTION FEEDBACK

Previous action executed successfully.

Here is the updated first-person view. Choose the next action (or DONE).

FAILED-ACTION FEEDBACK

Your action "grab wineglass" could not be grounded from this pose (invalid or unreachable).

The view did not change. Try a different next action.

ACTION HISTORY

Previous actions:

1. walk wineglass -> success

2. grab wineglass -> success

3. walk dishwasher -> success

4. putin wineglass dishwasher -> failed (invalid or unreachable)

Figure 8: VirtualHome direct-interaction prompt components and environment feedback. The current first-person image accompanies each request.